

Adrash (አድራሽ)

Build Specification — Agent-Ready Execution Plan

On-demand spare-parts delivery for Ethiopia — dispatch-first and built lean, benchmarked against Ride and Feres rather than an enterprise platform.

Version 2.0 · restructured from v1.1 for direct build execution · July 2026

1 Ground Rules for the Build Agent

Hand this document to a coding agent (Claude Code or similar) or to a developer and it should be enough to start Phase 1 with no scoping meeting. Read this section in full before writing any code.

- Build Phase 1 completely, deploy it, and confirm it works before opening Phase 2. Nothing from a later phase gets built early “for completeness.”
- Everything listed under **Explicitly deferred** is off-limits until its phase is reached — that includes new frameworks, extra services, admin dashboards, and anything ML-based.
- One deployable service for Phase 1. Not microservices, not a service mesh, not multiple repos.
- If a requirement below is ambiguous, ship the smallest version that works and leave a one-line comment noting the assumption. Do not silently add configuration options, edge-case handling, or abstraction layers nobody asked for.
- Rule of thumb for “is this too much”: would Ride or Feres have needed this to launch their first version in Addis Ababa? If not, it belongs in a later phase.

2 Product Summary

Adrash (አድራሽ) connects spare-parts vendors with independent motorbike drivers for on-demand delivery — the same dispatch model Ride and Feres use for passengers, applied to parts instead of people. A vendor requests a driver, the nearest available driver accepts, delivers, and both sides rate the interaction afterward. The build should feel as unremarkable and fast as booking one of those rides — that’s the bar, not a reason to add more.

3 Phase 1 — Build This Now

PHASE 1 — MVP

The only phase to build right now. Everything else waits.

3.1 Features

- **Vendor:** register with phone OTP, post a delivery request (item description, pickup pin, dropoff pin), track status, pay cash on delivery.
- **Driver:** register with phone OTP and an ID photo for manual staff review (no automated KYC service yet), toggle online/offline, see open requests near their last GPS ping, accept one at a time, mark picked-up and delivered.
- **Matching:** nearest available driver by straight-line distance from the last GPS ping — a plain SQL query over a lat/long bounding box. No PostGIS, no scoring model.

- **Notifications:** push notification when the app is active; SMS fallback for the three critical events — order confirmed, driver assigned, delivered.
- **Rating:** 1–5 stars in both directions with an optional comment. No dispute workflow yet.
- **Language:** Amharic and English from day one — this is core, not a later polish pass.

3.2 Tech Stack

Layer	Phase 1 choice	Notes
Frontend	React Native	One codebase, role-gated views for vendor and driver.
Backend	Node.js and Express	Single service. No separate microservices.
Database	PostgreSQL (managed)	Hosted (e.g. Render/Supabase). Don't self-manage a DB server.
Auth	Phone OTP, JWT and refresh token	Matches how people already sign into Ride and telebirr.
Hosting	Single instance	Render, Railway, or Fly.io. No load balancer or orchestration yet.
Push	FCM	Free tier covers MVP volume.
SMS	Any aggregator with Ethiopian carrier routes	Confirm coverage and pricing before committing — don't hard-code a vendor into the architecture.

3.3 Data Model

Entity	Key fields
users	id, phone (unique), role (vendor driver), name, language_pref (am en), created_at
driver_profiles	user_id (FK), id_photo_url, verification_status (pending approved rejected), is_online, last_lat, last_lng, last_ping_at
orders	id, vendor_id (FK), driver_id (FK, nullable), pickup_lat/lng, dropoff_lat/lng, item_description, status (requested matched picked_up delivered cancelled), created_at, matched_at, delivered_at
ratings	id, order_id (FK), rater_id (FK), ratee_id (FK), stars (1–5), comment, created_at

3.4 API Surface

Endpoint	Purpose	Caller
POST /auth/otp/request	Send OTP to a phone number	vendor, driver
POST /auth/otp/verify	Verify OTP; issue JWT + refresh token	vendor, driver
GET /me	Current user profile	vendor, driver
POST /orders	Create a delivery request	vendor
GET /orders/:id	Order detail and current status	vendor, driver

Endpoint	Purpose	Caller
GET /orders/nearby	Open orders within radius of driver's last ping	driver
POST /orders/:id/accept	Claim an order — requires an idempotency key	driver
POST /orders/:id/status	Update to picked_up or delivered	driver
POST /orders/:id/cancel	Cancel while still unmatched	vendor
POST /orders/:id/rate	Submit a 1–5 star rating	vendor, driver
POST /driver/ping	GPS heartbeat (lat, lng)	driver
POST /driver/online, /offline	Toggle availability for dispatch	driver

3.5 Order Lifecycle

requested -> matched -> picked_up -> delivered

requested -> cancelled (only while status = requested)

3.6 Explicitly Deferred — Do Not Build Yet

Off-limits until its phase is reached:

- Digital payments — Chapa, telebirr, CBE Birr, escrow -> Phase 2.
- Automated ID / KYC verification service -> Phase 2 (manual review queue for now).
- Dispute-resolution workflow -> Phase 2.
- Redis, message queue, WebSockets -> Phase 2 (polling every 5–8s is fine at MVP scale).
- PostGIS / geospatial indexing -> Phase 3.
- Monitoring/alerting stack, circuit breakers, disaster-recovery runbook -> Phase 3.
- Container orchestration, multi-service architecture -> Phase 3 at the earliest, and only if a single instance genuinely can't keep up.

4 Requirements That Apply to Every Phase

Carried forward from the original spec because they are cheap to build and directly serve how the platform will actually be used — they do not wait for a later phase.

- Amharic is a first-class UI language from day one, not a translation bolt-on added later.
- Every core screen works on 2G/3G: small payloads, infrequent polling (5–8s), minimal image and map-tile weight.
- Phone-based OTP only for login — no email/password flow. This matches how people already sign into Ride and telebirr.
- Cash-on-delivery is the default from day one — it must never be gated behind a payments integration.
- Client-side request queuing so an order survives a dropped connection and syncs once it returns. Cheap to build, and directly serves the low-connectivity reality outside Addis Ababa.

5 Security Essentials — Phase 1, Non-Negotiable

Unlike features, none of these wait for a later phase.

- TLS in transit; encryption at rest (on by default with most managed Postgres providers — verify, don't assume).
- Short-lived JWT access tokens plus refresh tokens; rate-limit OTP requests and login attempts. Redis isn't required for this at MVP scale — an in-process or DB-backed counter is enough until Phase 2's Redis lands.
- Secrets only in environment variables or a secrets manager — never hard-coded or committed.
- **Idempotency key required on POST /orders/:id/accept from day one.** A retried request must never double-assign the same order to two drivers. This is the one piece of “enterprise” security that isn't optional at MVP — the failure mode (two drivers show up for one order) breaks trust immediately.

6 Phase 2 — After Phase 1 Ships

PHASE 2 — GROWTH

Open only once Phase 1 is live and has real usage.

- Chapa integration for telebirr and CBE Birr; escrow-style hold released on delivery confirmation.
- Automated ID/KYC verification — or keep the manual queue if volume doesn't yet justify the integration cost.
- Redis for live driver location and rate-limit counters, off the primary database.
- Message queue (RabbitMQ or Redis Streams) decoupling order creation from driver matching.
- WebSockets (Socket.io) or push-driven live tracking, replacing polling.
- Lightweight dispute-resolution flow tied to the immutable delivery log.

7 Phase 3 — Scale Triggers

PHASE 3 — SCALE

Build only when the stated trigger is actually hit — not before.

- **PostGIS or geohashing** for nearby-driver queries — trigger: query latency creeps up, or the active-driver table passes roughly 5,000 rows.
- **Load balancer and horizontal auto-scaling**, containerized services — trigger: a single instance is consistently maxed on CPU or memory.
- **Monitoring and alerting** (Prometheus/Grafana or a hosted equivalent), circuit breakers around the matching queue, a documented backup/disaster-recovery procedure — trigger: real paying users depend on uptime, not just pilot users.
- **Formal automation sandboxing** — explicit allow-listed actions, hard timeouts, retry limits, a separate audit trail for automated actions — trigger: any dispatch or matching logic beyond simple nearest-driver rules is introduced.

8 Appendix — Where Everything From v1.1 Went

Nothing from the original specification was deleted. It was relocated to the phase where it's actually needed.

Original section (v1.1)	Where it lives now
1.1–1.2 Client and backend architecture	Phase 1 tech stack (trimmed); Redis, queue, and WebSockets moved to Phase 2; PostGIS moved to Phase 3
2.3 Payments	Phase 1 = cash-on-delivery only; Chapa, telebirr, CBE Birr and escrow moved to Phase 2
2.4 Low-connectivity resilience	Section 4 (applies from Phase 1); SMS fallback lands in Phase 2
2.5 Driver verification and trust	Phase 1 = phone OTP plus manual ID photo review; automated KYC and dispute flow moved to Phase 2
3. Security protocols	Section 5 (Phase 1 essentials); full automation sandboxing formalized in Phase 3
4. Reliability and observability	Phase 3, gated behind explicit scale triggers
5. Design and usability guidelines	Section 4 (applies to every phase)

Additions from v1.1 are preserved, not deleted — just placed where they're actually needed.